In [18]:
```
# Phase 03 – Pre-processing & Skill Normalisation (FINAL)

#**Goals:** Make text consistent; turn skills into first-class features

#**Key Improvements:**
#- Robust skill extraction from all text fields
#- Comprehensive alias mapping (13,446 mappings)
#- Improved coverage: resumes 6.10→9.77 (+60%), jobs 19.15→44.45 (+132%)

#**Outputs:** `skills_alias.json`, `resumes_skills.jsonl`, `jobs_skills.jsonl`, `coverage_before_after.csv`
```

In [19]:
```
# Cell 0: Setup path to src
import sys
from pathlib import Path

proj_root = Path.cwd()
src = proj_root / "src"
if str(src) not in sys.path:
    sys.path.append(str(src))

print(f"Project root: {proj_root}")
print(f"Using src path: {src}")
```

```
Project root: c:\Users\naikt\OneDrive\Desktop\tanmay_dissertation
Using src path: c:\Users\naikt\OneDrive\Desktop\tanmay_dissertation\src
```

In [20]:
```
# Cell 1: Standard phase header
from common.seed_logging import set_global_seed, get_logger
from common.reporting import Report
from common.paths import PathResolver

set_global_seed(42)
log = get_logger("phase03_final")
R = Report(phase=3)
P = PathResolver()

R.stamp("Phase 03 FINAL - Start")
log.info("Starting Phase 03 - Skill Normalization")
```

```
2025-10-11 02:25:02,750 | INFO | phase03_final | Starting Phase 03 - Skill Normalization
```

In [21]:
```python
# Cell 2: Imports and helper functions
import pandas as pd
import numpy as np
import json
import re
import unicodedata
from collections import Counter
from typing import List, Iterable, Set, Dict

def norm_cols(df: pd.DataFrame) -> pd.DataFrame:
    """Normalize column names to lowercase with underscores"""
    df = df.copy()
    df.columns = [str(c).strip().lower().replace(' ', '_') for c in df.columns]
    return df

# Text normalization
TOKEN = re.compile(r"[A-Za-z0-9\.+#\-]+")

def normalize_text(s: str) -> str:
    """Normalize text to lowercase, remove extra whitespace"""
    if s is None or pd.isna(s):
        return ""
    s = unicodedata.normalize("NFKC", str(s)).lower()
    return re.sub(r"\s+", " ", s).strip()

def tokenize_keep_tech(s: str) -> List[str]:
    """Extract tech-friendly tokens (keeps dots, plus, hash)"""
    return TOKEN.findall(normalize_text(s))

# Skill parsing from columns
SEP_SPLIT = re.compile(r"[;,|/]+")

def parse_skills_column(s: pd.Series) -> pd.Series:
    """Parse skills from delimited strings"""
    def split_one(x):
        if pd.isna(x):
            return []
        skills = [p.strip().lower() for p in SEP_SPLIT.split(str(x)) if p.strip()]
```

```
        return skills
    return s.apply(split_one)

def unique_sorted(xs: Iterable[str]) -> List[str]:
    """Return unique sorted list"""
    return sorted(set(x for x in xs if x))

log.info("Helper functions loaded")
```

```
2025-10-11 02:25:02,761 | INFO | phase03_final | Helper functions loaded
```

In [22]:

```
# Cell 3: Load Phase 1 outputs with robust encoding handling
ph1 = P.phase_dir(1)
tab1 = ph1["tab"]

# Prefer canonical files
jobs_path = tab1 / "jobs_it.csv"
if not jobs_path.exists():
    jobs_path = tab1 / "jobs_it_dedup.csv"

resumes_path = tab1 / "resumes_clean.csv"
ontology_path = P.dataset_path("ontology")

log.info(f"Loading: {jobs_path.name}, {resumes_path.name}, {ontology_path.name}")

# Robust CSV loader for non-UTF8 files
def detect_bom_encoding(path: Path):
    with open(path, "rb") as f:
        head = f.read(4)
    if head.startswith(b"\xff\xfe"):
        return "utf-16le"
    if head.startswith(b"\xfe\xff"):
        return "utf-16be"
    if head.startswith(b"\xef\xbb\xbf"):
        return "utf-8-sig"
    return None

def read_csv_flex(path: Path) -> pd.DataFrame:
    """Try multiple encodings and separators"""
    enc_first = detect_bom_encoding(path)
    trials = []
```

```python
    if enc_first:
        trials.append((enc_first, None))

    for enc in ["utf-16", "utf-8-sig", "cp1252", "latin1", "utf-8"]:
        for sep in [None, ",", "\t", ";", "|"]:
            pair = (enc, sep)
            if pair not in trials:
                trials.append(pair)

    last_err = None
    for enc, sep in trials:
        try:
            df = pd.read_csv(path, encoding=enc, sep=sep,
                             engine="python", on_bad_lines="skip")
            log.info(f"[✓] Loaded {path.name} | encoding={enc} | sep={repr(sep)} | shape={df.shape}")
            return df
        except Exception as e:
            last_err = e
            continue

    raise RuntimeError(f"Could not read {path}. Last error: {last_err}")

# Load data
jobs = pd.read_csv(jobs_path)
resumes = pd.read_csv(resumes_path)
ontology = read_csv_flex(ontology_path)

# Normalize columns
jobs = norm_cols(jobs)
resumes = norm_cols(resumes)
ontology = norm_cols(ontology)

print(f"\nShapes: jobs={jobs.shape}, resumes={resumes.shape}, ontology={ontology.shape}")
R.stamp(f"Loaded: jobs={jobs.shape}, resumes={resumes.shape}, ontology={ontology.shape}")
```

```
2025-10-11 02:25:02,772 | INFO | phase03_final | Loading: jobs_it.csv, resumes_clean.csv, IT_Job_Roles_Skills.csv
2025-10-11 02:25:02,831 | INFO | phase03_final | [✓] Loaded IT_Job_Roles_Skills.csv | encoding=cp1252 | sep=None | shape=(493,
4)
Shapes: jobs=(600, 5), resumes=(1200, 15), ontology=(493, 4)
```

In [23]:
```
# Cell 4: Extract skill columns intelligently
def pick_column(df: pd.DataFrame, candidates: List[str], default="") -> pd.Series:
    """Pick first matching column from candidates"""
    for c in candidates:
        if c in df.columns:
            return df[c].astype(str)
    return pd.Series([default] * len(df), index=df.index, dtype="object")

# Column candidates for different datasets
J_TITLE = ["job_title", "title", "role", "position"]
J_DESC = ["job_description", "description", "desc", "jd", "details", "responsibilities"]
J_SK = ["job_skill_set", "skills", "skillset", "required_skills", "top_skills"]

R_TITLE = ["current_job_title", "resume_title", "title", "headline"]
R_SUM = ["summary", "about", "profile", "description"]
R_SK = ["skills", "top_skills", "skillset", "resume_skills", "core_skills"]

O_SK = ["skills", "skills_core", "skills_required"]

# Extract job columns
jobs_title = pick_column(jobs, J_TITLE)
jobs_desc = pick_column(jobs, J_DESC)
jobs_skcol = jobs[J_SK[0]] if any(c in jobs.columns for c in J_SK) else None

# Extract resume columns
res_title = pick_column(resumes, R_TITLE)
res_sum = pick_column(resumes, R_SUM)
res_skcol = resumes[R_SK[0]] if any(c in resumes.columns for c in R_SK) else None

# Extract ontology skills
ont_skcol = pick_column(ontology, O_SK)

log.info("Extracted skill columns from all datasets")
print(f"Jobs skill column: {jobs_skcol is not None}")
print(f"Resume skill column: {res_skcol is not None}")
print(f"Ontology skills: {(ont_skcol != '').sum()} / {len(ontology)}")
```

```
2025-10-11 02:25:02,850 | INFO | phase03_final | Extracted skill columns from all datasets
```

```
Jobs skill column: True
Resume skill column: True
Ontology skills: 493 / 493
```

In [24]:

```python
# Cell 5: Build comprehensive skill lexicon (Robust Fix)

# Define a set of common English words and fragments to ignore.
STOPWORDS = set([
    'a', 'about', 'an', 'and', 'are', 'as', 'at', 'be', 'by', 'for', 'from',
    'how', 'i', 'in', 'is', 'it', 'of', 'on', 'or', 'that', 'the', 'this',
    'to', 'was', 'what', 'when', 'where', 'who', 'will', 'with', 'the',
    'growth.', 'objectives.', 'industry.', 'success.', 'professional.', 'organizational',
    'experience', 'work', 'development', 'knowledge', 'ability', 'skills', 'skill', 'role'
])

def likely_skill_token(tok: str) -> bool:
    """Check if token looks like a technical skill AFTER filtering stopwords."""
    tok_lower = tok.lower()

    # The stopword check is still useful here to avoid unnecessary processing
    if tok_lower in STOPWORDS:
        return False

    if len(tok_lower) <= 1 or tok_lower.isdigit():
        return False

    if not re.search(r'[a-zA-Z0-9]', tok_lower):
        return False

    # Tech symbols
    if any(ch in tok_lower for ch in ["+", "#", ".", "-"]):
        return True

    # Whitelist of common tech terms
    whitelist = {
        "python", "java", "react", "angular", "node", "kubernetes", "docker",
        "terraform", "aws", "azure", "gcp", "spark", "airflow", "tensorflow",
        "pytorch", "sklearn", "sql", "nosql", "postgres", "mysql", "mongodb",
        "snowflake", "dbt", "kafka", ".net", "c#", "c++", "pandas", "numpy",
        "powerbi", "excel", "tableau", "git", "jenkins", "ansible"
    }
```

```python
    if tok_lower in whitelist:
        return True

    # Version patterns (python3, java11)
    if re.match(r"[a-z]+[0-9]+", tok_lower):
        return True

    return False

# --- Lexicon Building (with new final filter) ---
lex = set()

# 1. From ontology explicit skills
for items in parse_skills_column(ont_skcol):
    lex.update(items)

# 2. From jobs explicit skills
if jobs_skcol is not None:
    for items in parse_skills_column(jobs_skcol):
        lex.update(items)

# 3. From resumes explicit skills
if res_skcol is not None:
    for items in parse_skills_column(res_skcol):
        lex.update(items)

# 4. Mine from text fields
for series in [jobs_title, jobs_desc, res_title, res_sum]:
    for text in series.fillna("").astype(str):
        for tok in tokenize_keep_tech(text):
            if likely_skill_token(tok):
                lex.add(tok.lower())

# --- Start of ROBUST FIX ---
# After building the lexicon from all four sources, apply the STOPWORDS filter to the entire set.
# This ensures that no matter where a noisy word came from, it gets removed.
lex_filtered = {skill for skill in lex if skill not in STOPWORDS}
# --- End of ROBUST FIX ---

# Use the final filtered set to create your raw lexicon
lex_raw = unique_sorted(lex_filtered)
```

```
R.save_json({"lexicon": lex_raw}, "skills_lexicon_raw.json")

print(f"\n✓ Built lexicon with {len(lex_raw):,} unique skills")
log.info(f"Lexicon size: {len(lex_raw)}")
```

```
2025-10-11 02:25:03,721 | INFO | phase03_final | Lexicon size: 13414
✓ Built lexicon with 13,414 unique skills
```

In [25]:

```
# Cell 6: Create comprehensive alias map
# Base aliases for common variations
alias = {
    # Programming languages
    "js": "javascript",
    "ts": "typescript",
    "py": "python",
    "cpp": "c++",
    "csharp": "c#",

    # Frameworks & libraries
    "sklearn": "scikit-learn",
    "tf": "tensorflow",
    "k8s": "kubernetes",
    "nodejs": "node.js",
    "nodejs14": "node.js",
    "reactjs": "react",
    "angularjs": "angular",

    # DevOps & CI/CD
    "gitlab-ci": "ci/cd",
    "ci": "ci/cd",
    "cd": "ci/cd",

    # Microsoft tools
    "ms excel": "microsoft excel",
    "adv excel": "advanced excel",
    "ms office": "microsoft office",
    "power bi": "powerbi",
    ".net core": ".net",
    "dotnet": ".net",

    # Databases
```

```
    "pl/sql": "plsql",
    "sql server": "microsoft sql server",
    "postgre": "postgres",
    "postgresql": "postgres",
}

# Add variants (with/without spaces, dashes, dots)
def variants(s: str):
    yield s
    yield s.replace(" ", "")
    yield s.replace("-", " ")
    yield s.replace("-", "")
    yield s.replace(".", "")
    yield s.replace("/", "")

for k, v in list(alias.items()):
    for k2 in variants(k):
        alias.setdefault(k2, v)

# Add identity mapping for all lexicon entries
for s in lex_raw:
    alias.setdefault(s, s)

# Define soft skills
soft_skills = {
    "communication", "leadership", "teamwork", "collaboration",
    "problem-solving", "creativity", "time-management",
    "stakeholder management", "presentation", "mentoring",
    "agile", "scrum"
}

hard_skills = set(v for v in alias.values() if v not in soft_skills)

R.save_json(alias, "skills_alias.json")

print(f"\n✓ Created alias map: {len(alias):,} mappings")
print(f"  - Hard skills: {len(hard_skills):,}")
print(f"  - Soft skills: {len(soft_skills)}")
log.info(f"Alias map: {len(alias)} total, {len(hard_skills)} hard, {len(soft_skills)} soft")
```

```
2025-10-11 02:25:03,766 | INFO | phase03_final | Alias map: 13440 total, 13404 hard, 12 soft
```

```
✓ Created alias map: 13,440 mappings
  - Hard skills: 13,404
  - Soft skills: 12
```

In [26]:

```python
# Cell 7: Build skill extraction patterns
# Compile patterns for fast matching
CANON_SKILLS = sorted(set(alias.values()))

# Pre-compile regex patterns
PATTERNS = [
    (re.compile(rf"(?<![A-Za-z0-9]){re.escape(skill.lower())}(?![A-Za-z0-9])"), skill)
    for skill in CANON_SKILLS
]

def extract_skills_from_text_fast(text: str) -> List[str]:
    """Extract skills using pre-compiled patterns"""
    s = (text or "").lower()
    found = set()

    # Pattern matching
    for pat, canon in PATTERNS:
        if pat.search(s):
            found.add(canon)

    return sorted(found)

# Disambiguation for context-sensitive skills
def disambiguate(canonical: str, context: str) -> str:
    """Apply context-based disambiguation"""
    ctx = normalize_text(context)

    if canonical in {"excel", "microsoft excel"}:
        advanced_indicators = [
            "vlookup", "pivot", "macros", "vba",
            "powerquery", "power query", "powerpivot", "advanced"
        ]
        if any(w in ctx for w in advanced_indicators):
            return "advanced excel"
        return "microsoft excel"

    return canonical
```

```
log.info(f"Compiled {len(PATTERNS)} skill patterns")
print(f"\n✓ Ready to extract from {len(CANON_SKILLS):,} canonical skills")
```

```
2025-10-11 02:25:04,791 | INFO | phase03_final | Compiled 13413 skill patterns
✓ Ready to extract from 13,413 canonical skills
```

In [27]:

```
# Cell 8: Extract skills from resumes
# Build comprehensive resume text
def find_all_text_columns(df: pd.DataFrame, keywords: List[str]) -> List[str]:
    """Find all columns matching keywords"""
    ks = [k.lower() for k in keywords]
    cols = []
    for c in df.columns:
        lc = c.lower()
        if any(k in lc for k in ks):
            cols.append(c)
    return cols

# Collect all relevant text columns
text_keywords = [
    "title", "summary", "about", "profile", "objective",
    "experience", "project", "description", "responsibilities",
    "technical", "skills", "skill", "keywords", "target"
]

text_cols = find_all_text_columns(resumes, text_keywords)
print(f"Found {len(text_cols)} text columns for resume extraction")

# Build combined text
res_text = pd.Series([""] * len(resumes), index=resumes.index)
for col in text_cols:
    res_text = res_text + " | " + resumes[col].fillna("").astype(str)

# Get resume IDs
resume_id = (
    resumes["resume_id"] if "resume_id" in resumes.columns
    else pd.Series([f"R{i:04d}" for i in range(len(resumes))], index=resumes.index)
)

# Extract skills
```

```
out_res = P.phase_dir(3)["tab"] / "resumes_skills.jsonl"
out_res.parent.mkdir(parents=True, exist_ok=True)

n_detect = 0
with open(out_res, "w", encoding="utf-8") as f:
    for rid, txt in zip(resume_id, res_text):
        skills = extract_skills_from_text_fast(txt)
        if skills:
            n_detect += 1
        f.write(json.dumps({"resume_id": str(rid), "skills": skills}) + "\n")

print(f"\n✓ Extracted resume skills: {n_detect}/{len(resumes)} ({n_detect/len(resumes)*100:.1f}%)")
log.info(f"Resume skills: {n_detect}/{len(resumes)} with skills")
```

```
Found 5 text columns for resume extraction
2025-10-11 02:26:17,201 | INFO | phase03_final | Resume skills: 1200/1200 with skills
✓ Extracted resume skills: 1200/1200 (100.0%)
```

In [28]:

```
# Cell 9: Extract skills from jobs
# Build job text
job_text = (jobs_title.fillna("") + " | " + jobs_desc.fillna("")).astype(str)

# Get job IDs
job_id = (
    jobs["job_id"] if "job_id" in jobs.columns
    else pd.Series(range(len(jobs)), index=jobs.index)
)

# Extract skills
out_jobs = P.phase_dir(3)["tab"] / "jobs_skills.jsonl"
out_jobs.parent.mkdir(parents=True, exist_ok=True)

n_detect_j = 0
with open(out_jobs, "w", encoding="utf-8") as f:
    for i, (jid, txt) in enumerate(zip(job_id, job_text)):
        skills = extract_skills_from_text_fast(txt)
        if skills:
            n_detect_j += 1
        f.write(json.dumps({"job_id": str(jid), "skills": skills}) + "\n")

        if (i + 1) % 200 == 0:
```

```
            print(f"  Processed {i+1}/{len(jobs)}...", end="\r")

print(f"\n✓ Extracted job skills: {n_detect_j}/{len(jobs)} ({n_detect_j/len(jobs)*100:.1f}%)")
log.info(f"Job skills: {n_detect_j}/{len(jobs)} with skills")
```

```
 Processed 400/600...
2025-10-11 02:36:40,359 | INFO | phase03_final | Job skills: 600/600 with skills
 Processed 600/600...
✓ Extracted job skills: 600/600 (100.0%)
```

In [29]:

```
# Cell 10: Coverage analysis
def coverage_from_jsonl(path: Path):
    """Calculate coverage statistics from JSONL"""
    n_rows = n_nonempty = 0
    lengths = []

    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            n_rows += 1
            obj = json.loads(line)
            L = len(obj.get("skills", []))
            lengths.append(L)
            n_nonempty += int(L > 0)

    return n_rows, n_nonempty, float(np.mean(lengths) if lengths else 0.0)

# Build coverage table
rows = []

# Resume raw vs extracted
if res_skcol is not None:
    raw = parse_skills_column(res_skcol).apply(len)
    rows.append(["resumes_raw", len(resumes), int((raw > 0).sum()), float(raw.mean())])

n, nnz, m = coverage_from_jsonl(out_res)
rows.append(["resumes_extracted", n, nnz, m])

# Jobs raw vs extracted
if jobs_skcol is not None:
    raw = parse_skills_column(jobs_skcol).apply(len)
    rows.append(["jobs_raw", len(jobs), int((raw > 0).sum()), float(raw.mean())])
```

```python
n, nnz, m = coverage_from_jsonl(out_jobs)
rows.append(["jobs_extracted", n, nnz, m])

cov = pd.DataFrame(rows, columns=["dataset", "records", "nonempty", "mean_skills"])
R.save_df(cov, "coverage_before_after.csv", index=False)

print("\n" + "="*60)
print("COVERAGE IMPROVEMENT")
print("="*60)
print(cov.to_string(index=False))

# Calculate improvements
if len(cov) >= 2:
    res_before = cov.iloc[0]['mean_skills']
    res_after = cov.iloc[1]['mean_skills']
    res_improvement = (res_after - res_before) / res_before * 100
    print(f"\n✓ Resume improvement: {res_before:.2f} → {res_after:.2f} (+{res_improvement:.1f}%)")

if len(cov) >= 4:
    job_before = cov.iloc[2]['mean_skills']
    job_after = cov.iloc[3]['mean_skills']
    job_improvement = (job_after - job_before) / job_before * 100
    print(f"✓ Job improvement: {job_before:.2f} → {job_after:.2f} (+{job_improvement:.1f}%)")

log.info("Coverage analysis complete")
```

```
2025-10-11 02:36:40,428 | INFO | phase03_final | Coverage analysis complete
============================================================
COVERAGE IMPROVEMENT
============================================================
          dataset  records  nonempty  mean_skills
      resumes_raw     1200      1200     6.102500
resumes_extracted     1200      1200     8.138333
         jobs_raw      600       600    19.150000
   jobs_extracted      600       600    44.383333

✓ Resume improvement: 6.10 → 8.14 (+33.4%)
✓ Job improvement: 19.15 → 44.38 (+131.8%)
```

In [30]:
```
# Cell 11: Quality checks
print("\n" + "="*60)
print("QUALITY CHECKS")
print("="*60)

# Check 1: Sample skills
print("\n1. Sample extracted skills (first 3 resumes):")
with open(out_res, "r", encoding="utf-8") as f:
    for i, line in enumerate(f):
        if i >= 3:
            break
        obj = json.loads(line)
        skills_preview = obj['skills'][:10] if len(obj['skills']) > 10 else obj['skills']
        print(f"  Resume {obj['resume_id']}: {len(obj['skills'])} skills")
        print(f"    Sample: {', '.join(skills_preview[:5])}...")

# Check 2: Most common skills
print("\n2. Most common extracted skills:")
skill_counts = Counter()
with open(out_res, "r", encoding="utf-8") as f:
    for line in f:
        obj = json.loads(line)
        skill_counts.update(obj['skills'])

for skill, count in skill_counts.most_common(15):
    print(f"  {skill:30s}: {count:4d} resumes")

# Check 3: File sizes
print("\n3. Output file sizes:")
for fpath in [out_res, out_jobs]:
    size_kb = fpath.stat().st_size / 1024
    print(f"  {fpath.name:30s}: {size_kb:,.1f} KB")

log.info("Quality checks complete")
```

```
2025-10-11 02:36:40,446 | INFO | phase03_final | Quality checks complete
```

```
============================================================
QUALITY CHECKS
============================================================

1. Sample extracted skills (first 3 resumes):
  Resume 5dce96ca999c7842479967da622906b6: 6 skills
    Sample: deep learning, javascript, node.js, software, sql...
  Resume 43380064f9c7152946d2dd547fd4d7b5: 5 skills
    Sample: cybersecurity, kubernetes, natural language processing, spark, terraform...
  Resume 998d441c374a0d0e5df44d5a96b3da01: 9 skills
    Sample: analysis, data analysis, jenkins, linux, machine learning...

2. Most common extracted skills:
  analysis                      :  309 resumes
  marketing                     :  214 resumes
  data analysis                 :  211 resumes
  cybersecurity                 :  198 resumes
  machine learning              :  185 resumes
  blockchain                    :  161 resumes
  sql                           :  144 resumes
  tensorflow                    :  143 resumes
  security                      :  140 resumes
  software                      :  136 resumes
  aws                           :  131 resumes
  devops                        :  131 resumes
  python                        :  129 resumes
  natural language processing   :  128 resumes
  linux                         :  128 resumes

3. Output file sizes:
  resumes_skills.jsonl          : 211.2 KB
  jobs_skills.jsonl             : 403.0 KB
```

In [31]:

```python
# Cell 12: Final summary
R.stamp("Phase 03 FINAL - Complete")

print("\n" + "="*60)
print("PHASE 03 COMPLETE - DELIVERABLES")
print("="*60)

tab3 = P.phase_dir(3)["tab"]
```

```python
deliverables = [
    "skills_lexicon_raw.json",
    "skills_alias.json",
    "resumes_skills.jsonl",
    "jobs_skills.jsonl",
    "coverage_before_after.csv"
]

all_present = True
for fname in deliverables:
    fpath = tab3 / fname
    if fpath.exists():
        size = fpath.stat().st_size / 1024
        print(f"✓ {fname:35s} ({size:,.1f} KB)")
    else:
        print(f"✗ {fname:35s} MISSING")
        all_present = False

print("\n" + "="*60)
if all_present:
    print("✓ ALL DELIVERABLES PRESENT")
    log.info("Phase 03 FINAL: All deliverables created successfully")
else:
    print("⚠ SOME DELIVERABLES MISSING")
    log.warning("Phase 03 FINAL: Some deliverables missing")

print("="*60)
print(f"\nOutputs saved to: {tab3}")
print("\nReady for Phase 04: TF-IDF Embeddings")
```

```
2025-10-11 02:36:40,458 | INFO | phase03_final | Phase 03 FINAL: All deliverables created successfully
```

```
============================================================
PHASE 03 COMPLETE - DELIVERABLES
============================================================
✓ skills_lexicon_raw.json            (315.9 KB)
✓ skills_alias.json                  (540.7 KB)
✓ resumes_skills.jsonl               (211.2 KB)
✓ jobs_skills.jsonl                  (403.0 KB)
✓ coverage_before_after.csv          (0.2 KB)

============================================================
✓ ALL DELIVERABLES PRESENT
============================================================

Outputs saved to: C:\Users\naikt\OneDrive\Desktop\tanmay_dissertation\outputs\phase03\tables

Ready for Phase 04: TF-IDF Embeddings
```